

Geo-Automation Framework

Technical Documentation & Codebase Handover Guide

Application under test	Geo Travel web app (gowithgeo.com)
Framework type	Selenium + pytest, Page Object Model, UI + REST API coverage
Prepared by	John · QA Automation
Date	September 1, 2026
Repository	geo-travel-automation

Contents

1.	Purpose & Scope	
2.	Technology Stack	
3.	Architecture Overview	
4.	Repository Layout	
5.	Environment & Configuration	
6.	Core Framework Reference	
7.	UI Page Object Reference	
8.	API Client Reference	
9.	Notifications & Reporting	
10.	Running the Suite	
11.	CI/CD Integration	
12.	Issues Found & Fixed	
13.	Remaining Recommendations for the Incoming Maintainer	

Purpose & Scope

The Geo-Automation Framework is the QA automation suite for the Geo Travel web application. It provides end-to-end UI coverage (via Selenium, driving a real browser through booking flows) and REST API coverage (via `requests`, exercising the retail and partner/B2B backends directly). Tests run locally, on demand, and on a daily CircleCI schedule, with results surfaced as HTML/Allure reports and Slack notifications.

This document explains the codebase end to end for a new maintainer — architecture, configuration, every core class and page object, the API client layer, and how test results reach the team (Section 9, Notifications & Reporting). While preparing it, a number of bugs were found across the codebase; the ones with unambiguous fixes were corrected directly in the source (Section 12), and the handful that are really product/contract decisions rather than bugs were left open and are listed separately (Section 13).

Documentation Scope

This pass covers the framework's architectural backbone — core classes, page objects, and API clients — at a level suitable for getting oriented without reading all ~19,000 lines directly. It does not re-document `src/tests/*` (the test suites themselves) or line-by-line internals of `src/utils/*` beyond what's needed to explain reporting and notifications.

AREA	FILES	COVERED HERE
Core framework (<code>src/core/</code> , <code>configs/</code>)	7	Section 6 (reference) + Section 12 (fixes)
UI page objects (<code>src/pages/ui/</code>)	11	Section 7 (reference) + Section 12 (fixes)
API clients (<code>src/pages/api/</code> , incl. <code>partners_api/</code>)	19	Section 8 (reference) + Section 12 (fixes)
Reporting & notifications (<code>src/utils/reporting.py</code> , <code>notifications.py</code>)	2	Section 9 (new, dedicated section)
Test suites (<code>src/tests/</code>)	~19	Referenced, not re-documented line by line

Technology Stack

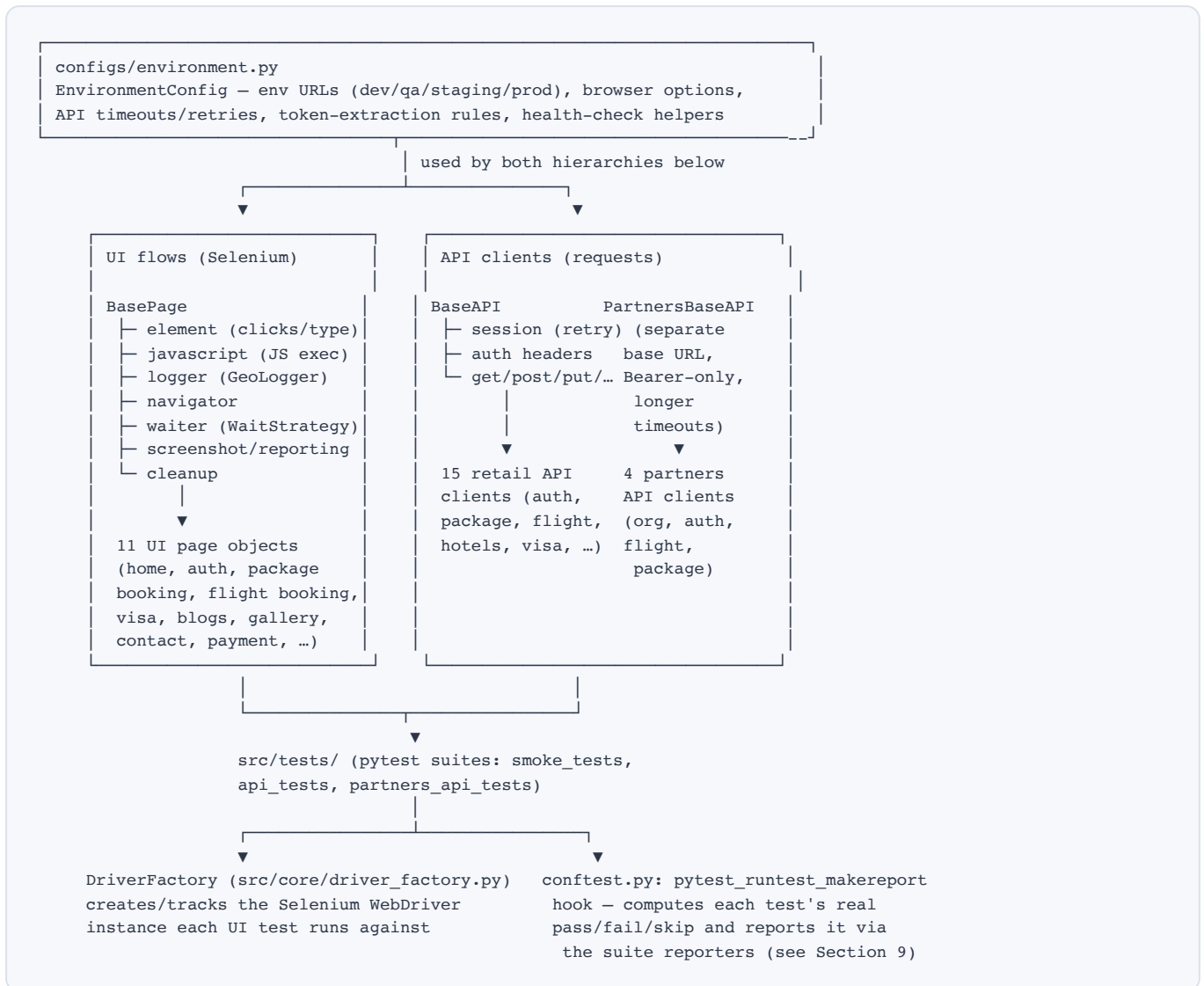
CONCERN	TOOL / LIBRARY
Browser automation	Selenium WebDriver (Chrome, Firefox, Edge via <code>webdriver-manager</code>)
Test runner	pytest, with <code>pytest-html</code> for self-contained HTML reports and Allure for richer reporting
HTTP client (API tests)	<code>requests</code> , wrapped in a retrying session (<code>BaseAPI</code> / <code>PartnersBaseAPI</code>)
Config & secrets	<code>.env</code> file loaded via <code>python-dotenv</code> , read through <code>EnvironmentConfig</code>
CI/CD	CircleCI (scheduled daily smoke run) + GitHub Actions (<code>qa-tests.yml</code> , <code>autoassign.yml</code>)
Notifications	Slack (webhook + SDK fallback) for run/summary messages, SMTP email as a secondary channel — see Section 9
Code quality tooling	black, flake8, isort, bandit, mypy-extensions (present in <code>requirements.txt</code>)

Key Conventions Worth Knowing

- **Method chaining:** most UI page-object methods return `self` so steps can be chained (`navigate_to_packages().select_first_package()`).
- **Locators as class attributes:** every page object declares its Selenium locators as `(By.X, "...")` tuples at the top of the class, grouped by flow step under `# ===== SECTION =====` comments — locators are data, methods are behavior.
- **Cross-page coupling:** `package_booking_flow.py` imports and uses `PaymentPage` from `payment_flow.py` to verify checkout redirects. `flight_booking_flow.py` does not do this yet (see Section 13) — worth aligning if payment verification is extended to flights.
- **Two independent API base classes:** `BaseAPI` and `PartnersBaseAPI` intentionally duplicate most of their retry/session logic rather than sharing an ancestor. Their retry semantics have now been aligned (both re-raise once retries are exhausted — see Section 12), but the classes themselves are still separate; consolidating them behind a shared base remains a Section 13 recommendation.
- **Logging:** every class gets a `GeoLogger` instance scoped to its own class name (`self.logger = GeoLogger(self.__class__.__name__)`), used throughout for step-by-step traceability in reports.

Architecture Overview

The framework follows the Page Object Model (POM): every user-facing flow or API resource is wrapped in a class that exposes intention-revealing methods (`login()` , `search_packages()`) instead of raw Selenium/HTTP calls. Two parallel class hierarchies exist — one for browser-driven UI flows, one for HTTP API clients — because the underlying I/O (WebDriver vs. `requests.Session`) is fundamentally different, but both follow the same shape: a shared base class owns cross-cutting concerns (logging, config, retries/waits), and each concrete class owns only the locators/endpoints and steps specific to its flow.



Repository Layout

```

.
├── .circleci/config.yml      Scheduled daily smoke-test pipeline
├── .github/workflows/       qa-tests.yml, auto-assign.yml
├── .env                     Local secrets/config (not committed)
├── conftest.py              pytest fixtures + the makereport hook shared
                             across all suites (see Section 9)
├── pytest.ini               pytest config: discovery rules, markers, HTML report
├── configs/
│   └── environment.py       EnvironmentConfig – env URLs, browser & API config
├── scripts/
│   ├── run_scheduled_tests.py CLI entry point used by CI (--suite smoke/api/...)
│   └── cleanup.py           Prunes old reports/screenshots
├── src/
│   ├── core/               Base classes: BasePage, BaseAPI, PartnersBaseAPI,
│                           DriverFactory, TestBase
│   ├── pages/
│   │   ├── ui/             11 Selenium page objects
│   │   └── api/            15 retail API clients + partners_api/ (4 more)
│   ├── tests/
│   │   ├── smoke_tests/    UI smoke suites, one per major flow
│   │   ├── api_tests/      Retail API test suites
│   │   └── partners_api_tests/ Partner/B2B API test suites
│   └── utils/              Shared helpers: logger, waits, screenshots, JS
                             execution, notifications, reporting, etc.
├── reports/                Generated HTML/Allure test reports
└── logs/                   Run logs

```

Environment & Configuration

`configs/environment.py` defines `EnvironmentConfig`, a classmethod-based config hub (no instantiation needed) that switches behavior based on the `TEST_ENV` variable. It supports four named environments:

ENVIRONMENT	RETAIL BASE URL	RETAIL API
dev	retail.dev.gowithgeo.com	api.dev.gowithgeo.com
qa (default)	retail.qa.gowithgeo.com	api.qa.gowithgeo.com
staging	retail.stg.gowithgeo.com	api.stg.gowithgeo.com
production	www.gowithgeo.com	api.gowithgeo.com

Partner/B2B API URLs are configured separately per environment (sandbox URLs for dev/qa, distinct staging/production partner hosts).

Required .env variables

Names only — values are environment-specific secrets and are not reproduced here.

TEST_ENV	dev / qa / staging / production (default: qa)
BROWSER	chrome / firefox / edge (default: chrome)
HEADLESS	true/false
WINDOW_SIZE	e.g. 1920x1080
TIMEOUT	default Selenium wait timeout (seconds)
API_TIMEOUT / API_MAX_RETRIES	API client request tuning
API_TEST_EMAIL / API_TEST_PASSWORD	Retail API test account credentials
TEST_USER_EMAIL / TEST_USER_PASSWORD	UI login test account credentials
PARTNERS_VERIFIED_EMAIL / PARTNERS_VERIFIED_PASSWORD	Partners API verified test account
SLACK_WEBHOOK_URL / SLACK_BOT_TOKEN / SLACK_CHANNEL	Slack notification target — see Section 9
SMTP_SERVER / SMTP_PORT / EMAIL_USERNAME / EMAIL_PASSWORD / EMAIL_TO	Fallback email notification channel — see Section 9
PROJECT_NAME / BRANCH / BUILD_ID / ENVIRONMENT	Metadata surfaced in reports/notifications

Fixed: `EnvironmentConfig.get_api_credentials()` now reads `API_TEST_EMAIL / API_TEST_PASSWORD` from the environment as proper class attributes. Previously it referenced two class attributes that were never defined and always raised `AttributeError` when called. See Section 12.

Core Framework Reference

FILE	RESPONSIBILITY
src/core/base_page.py	<code>BasePage</code> — superclass for all Selenium UI page objects. Wires up <code>self.element</code> , <code>self.javascript</code> , <code>self.logger</code> , <code>self.navigator</code> , <code>self.pageinfo</code> , <code>self.reporting</code> , <code>self.screenshot</code> , <code>self.validator</code> , <code>self.waiter</code> , <code>self.cleanup</code> — every page object inherits this shared toolkit.
src/core/base_api.py	<code>BaseAPI</code> — superclass for retail API clients. Owns a <code>requests.Session</code> , manages auth headers/cookies (cookie-based or Authorization-header token styles), and exposes <code>get/post/put/patch/delete()</code> , all routed through a retrying <code>_request()</code> (3 attempts). Outgoing header logging is now redacted before it's written (see Section 12).
src/core/partners_base_api.py	<code>PartnersBaseAPI</code> — parallel superclass for the 4 Partners/B2B API clients. Separate base URL, Bearer-token-only auth, longer timeouts, and dumps failed response bodies to disk for easier debugging. Retry handling now consistently re-raises the underlying exception once attempts are exhausted, for every retried error type (see Section 12).
src/core/driver_factory.py	<code>DriverFactory</code> — creates and tracks Selenium WebDriver instances per the configured browser (chrome/firefox/edge), applying the options built by <code>EnvironmentConfig</code> .
src/core/test_base.py	<code>TestBase</code> — an optional pytest mixin adding setup/teardown timing, automatic failure screenshots, and smoke-test result reporting. Now correctly reports PASS/FAIL/SKIP based on the test's real outcome (see Section 12 — this was previously always reporting PASS).
src/core/__init__.py	Re-export surface for the package (exposes <code>BasePage</code> , <code>DriverFactory</code> at the package level).
configs/environment.py	<code>EnvironmentConfig</code> — central config hub described in Section 5.

UI Page Object Reference

FILE	FLOW COVERED
home_page.py	Homepage warm-up/load with retry logic, logo check, React/Next.js health check, and content-confidence validation that the right app loaded.
auth_flow.py	Login (email + password) and logout, including dashboard-indicator detection and toast-message capture.
dashboard_page.py	Post-login dashboard load and identity checks; wraps an <code>AuthFlow</code> instance to reuse its dashboard-indicator and logout locators/helpers.
package_booking_flow.py	Full package-booking journey: search (trip type, country, date) → select a price option → fill booking modal → log in → accept terms → choose a payment method → verify checkout destination (Flutterwave / Paystack / bank transfer). The most complete and heavily-used flow in the suite.
flight_booking_flow.py	One-way flight search (route currently hardcoded to Heathrow→Schiphol), result selection, passenger form, and payment-method click. Does not yet reuse <code>PaymentPage</code> or verify the checkout destination the way <code>package_booking_flow.py</code> does — see Section 13.
payment_flow.py	<code>PaymentPage</code> — triggers the Flutterwave hosted checkout and performs a minimal iframe-readiness check. Does not complete a full card payment end-to-end — flagged as a deliberate open question in Section 13 rather than fixed outright, since it changes what the flow actually exercises against a live payment gateway.
visa_enquiries_flow.py	Visa application form: personal details, country/passport/visa-type dropdowns, travel date picker, submit/cancel. <code>fill_personal_details()</code> now actually uses the name/email/phone passed in, <code>select_travel_date()</code> now targets the requested day (falling back to the old heuristic if it isn't available), and <code>click_get_started()</code> now raises instead of silently returning <code>None</code> after exhausting retries — see Section 12.
blogs_flow.py	Blog listing, detail page, comment submission, and "More like this" related posts.
travel_gallery_flow.py	Gallery listing, tour detail navigation, hover-triggered image viewer modal. <code>verify_gallery_has_tours()</code> now lets a "no tours found" assertion actually propagate instead of swallowing it (see Section 12).
contact_flow.py	Contact Support form submission, FAQ link, Privacy Statement link. <code>submit_form()</code> now retries up to <code>max_retries</code> times, as its signature always implied (see Section 12).
__init__.py	Empty package marker.

API Client Reference

Retail API (src/pages/api/) — subclass BaseAPI

FILE	ENDPOINT PREFIX	COVERS
auth_api.py	/api/auth	Login (with dynamic token extraction), logout, token refresh.
package_api.py	/api/package	Listing, detail, booking, deals, a user's booked packages/analytics.
flight_api.py	/api/flight	Search request/results, booking, quotes, passenger email validation. Query parameters are now passed via <code>params=</code> rather than interpolated into the URL — see Section 12.
flight_utils_api.py	/api/flight	Reference-data lookups: airports, airlines.
hotels_api.py	/api/hotels	Search, cities, ratings, booking, priced offers.
visa_enquiries_api.py	/api/visa	Create enquiry, pay, verify payment, list/lookup applications.
commercial_deals_api.py	/api/commercial	Deal listing, booking, payment verification.
transactions_api.py	/api/transactions	User transaction list, insights, CSV export.
users_api.py	/api/users/me	Read/update the logged-in user's profile.
notification_api.py	/api/notifications	List, mark-read, delete in-app notifications (distinct from the Slack/email test notifications in Section 9).
newsletter_api.py	/api/newsletter	Newsletter signup.
price_api.py	/api/price/voucher	Voucher lookup and apply.
blogs_api.py	/api/blog	Listing/fetching posts, reading/creating comments.
events_api.py	/api/event	List/fetch events.
google_reviews_api.py	/api/google/reviews	Surfaces Google reviews for the site.

Partners / B2B API (src/pages/api/partners_api/) — subclass PartnersBaseAPI

FILE	ENDPOINT PREFIX	COVERS
partners_auth_api.py	/api/auth	Signup, login (with token extraction), email verification, password reset.
organization_api.py	/api/org	Organization profile, API key reset, usage reporting.
partners_flight_api.py	/api/flight	Search/book/list bookings, using per-call API-key auth. Header logging redacts <code>x-api-secret</code> and is now debug-level — see Section 12.
partners_package_api.py	/api/package	List/book packages, countries, bookings, using per-call API-key auth.

Notifications & Reporting

Two systems work together to turn a test run into something the team actually sees: `src/utils/reporting.py` collects results and formats messages, and `src/utils/notifications.py` delivers them.

How a result gets recorded

`conftest.py` registers a single `pytest_runtest_makereport` hookwrapper that runs for every test, regardless of which base class (if any) it uses. For each test's "call" phase, it:

- Reads pytest's own outcome (`report.passed` / `.skipped` / otherwise failed) and builds a `PASS` / `FAIL` / `SKIP` status plus an error message.
- On failure, captures a screenshot (UI tests) or dumps the last API response body to `reports/failed_responses/` (API tests), and attaches the most recent log file to the pytest-html report.
- Routes the result to the right suite reporter via `get_suite_reporter()` (smoke, regression, sanity, api, partners_api, ...), each a `GeoReporter` instance from `reporting.py`.
- Stashes the computed status/error onto the test instance (`_test_outcome_status` / `_test_outcome_error`) so that test classes using the optional `TestBase` mixin (Section 6) can read the real outcome in their own `teardown_method` instead of guessing.

How a result becomes a message

`GeoReporter` accumulates results for a suite, then formats a rich Slack message once the run finishes:

- **Run-start message** — sent when a suite (or a unified multi-suite run) kicks off, with scope (frontend/backend/mixed) and start time.
- **Run-summary message** — pass/fail/skip counts, a per-failure breakdown (category, context, expected-vs-actual, evidence links to screenshots/logs/response dumps hosted on GitHub Pages, and a suggested fix where one can be inferred), and up to 5 skipped tests with their skip reason.
- **Environment-down alert** — sent by `scripts/run_scheduled_tests.py` when a pre-run health check (see `EnvironmentConfig.is_api_accessible` / `_is_ui_accessible`, Section 5) finds the target environment unreachable, so the whole run is skipped rather than failing every test individually.

Delivery channels (`src/utils/notifications.py`)

CLASS	CHANNEL	BEHAVIOR
SlackNotifier	Slack	<code>send_webhook_message()</code> tries the incoming webhook (<code>SLACK_WEBHOOK_URL</code>) first for richer formatting, and falls back to the Slack SDK (<code>SLACK_BOT_TOKEN</code> + <code>SLACK_CHANNEL</code> , default <code>#geo-automation</code>) if the webhook is unset or the request fails. <code>validate_notification_setup()</code> can be used to smoke-test the webhook independently.
EmailNotifier	SMTP	Secondary channel via <code>smtplib</code> , configured through <code>SMTP_SERVER</code> / <code>SMTP_PORT</code> / <code>EMAIL_USERNAME</code> / <code>EMAIL_PASSWORD</code> / <code>EMAIL_TO</code> . Used for <code>send_test_report()</code> -style summaries where email is preferred over Slack.

Operational note: both channels degrade gracefully when unconfigured — a missing `SLACK_WEBHOOK_URL` is logged as an error but does not fail the run, and the SDK fallback only engages if a bot token is present. If notifications silently stop appearing, check `SLACK_WEBHOOK_URL` / `SLACK_BOT_TOKEN` validity before assuming a code regression.

Running the Suite

```
# Install dependencies
pip install -r requirements.txt

# Run everything
pytest

# Run a specific suite via the CI helper script
python scripts/run_scheduled_tests.py --suite smoke
python scripts/run_scheduled_tests.py --suite api
python scripts/run_scheduled_tests.py --suite partners_api
python scripts/run_scheduled_tests.py --suite api smoke partners_api
```

Test discovery, markers (`smoke` , `regression` , `sanity` , `api` , `partners_api` , `security` , `auth` , `cross_browser`), and HTML report generation are configured centrally in `pytest.ini` . Reports land in `reports/` , logs in `logs/` ; `scripts/cleanup.py` prunes artifacts older than a configurable retention window.

Pass `--skip-env-check` to bypass the pre-run environment health check (useful when collecting tests locally without network access to the target environment) and `--env-timeout <seconds>` to tune how long that check waits.

CI/CD Integration

- **CircleCI** (`.circleci/config.yml`) runs a daily scheduled smoke + API test job at 07:00 UTC (08:00 Nigeria time), storing screenshots as artifacts and cleaning up test artifacts older than 7 days afterward. A regression-test job exists in the config but is currently commented out.
- **GitHub Actions** (`.github/workflows/`) runs `qa-tests.yml` and `auto-assign.yml` (the latter auto-assigns reviewers/comments on issues and PRs).
- **Slack** notifications are sent for run-start, run-summary, and environment-down alerts — see Section 9 for the full mechanism.

Issues Found & Fixed

The following bugs were found incidentally while writing docstrings across the framework's core classes, page objects, and API clients. The security issue and the large majority of the reliability/code-quality items have been fixed directly in the source as part of this same pass. Each entry below is also reflected as an updated docstring/comment at its location in the code.

SECURITY Secrets no longer logged in plaintext

```
src/core/base_api.py - _request()
```

Outgoing header logging (which included live `Authorization` / `Cookie` values) is now redacted via a `_redact_headers()` helper and moved to `debug` level, so tokens no longer land in plaintext in logs/CI output on every retail API request. The dead `self._debug_token_source` safety check (an attribute nothing ever set) was also fixed to read the real `self.token_source` attribute, so it now actually fires when an Authorization header is unexpectedly present alongside a cookie-based token.

```
src/pages/api/partners_api/partners_flight_api.py - _get_flight_headers()
```

The `x-api-secret` value is now masked before logging, and the logging itself moved to `debug` level — closing the same class of plaintext-secret-in-logs issue for Partners flight-API calls.

RELIABILITY Bugs that undermined trust in test results

```
src/core/test_base.py + conftest.py - TestBase.teardown_method()
```

Previously read `self._test_outcome`, an attribute nothing in the repo ever set — so any test using the `TestBase` mixin always reported "PASS", regardless of actual outcome. `conftest.py`'s existing `pytest_runtest_makereport` hook now stashes the real, already-computed status/error onto the test instance, and `teardown_method` reads that. `mark_test_failed()` can also now genuinely force a FAIL, which it previously could not. Verified end-to-end with a throwaway failing/passing test pair before and after the fix.

```
configs/environment.py - get_api_credentials()
```

Referenced two class attributes that were never defined, always raising `AttributeError`. `API_TEST_EMAIL` / `API_TEST_PASSWORD` are now read from the environment at class-definition time, matching the pattern used elsewhere in the class.

```
src/pages/ui/package_booking_flow.py - is_price_option_available()
```

Iterated `self.PRICE_OPTIONS`, which didn't exist. Now iterates `self.PRICE_OPTION_BY_TYPE.values()`, the dict that actually holds the locators.

```
src/pages/ui/dashboard_page.py - wait_for_dashboard_load()
```

Passed a list of locators into helpers that expect a single tuple. Now delegates to `AuthFlow.wait_for_any_dashboard_indicator()`, which already correctly iterates the list one locator at a time. The unused, invalid `LOGOUT_BUTTON` locator (a jQuery-only `:contains()` pseudo-class inside native CSS) was removed rather than fixed, since a working logout locator already exists on `AuthFlow`.

```
src/pages/ui/home_page.py - wait_for_homepage_load()
```

Raised `AssertionError(f"...: {e}")` where `e` was undefined in that branch, producing `NameError` instead of the intended assertion whenever validation failed on the final retry. Now builds the message from the actual confidence score instead.

```
src/pages/ui/visa_enquiries_flow.py - click_get_started()
```

Fell through to an implicit `None` after exhausting retries, breaking any chained call. Now raises the last attempt's exception instead, so a caller sees a clear failure rather than an `AttributeError` on `None`.

```
src/pages/ui/travel_gallery_flow.py - verify_gallery_has_tours()
```

Its own "no tours found" assertion was inside the surrounding `try/except Exception`, so it was always swallowed and logged rather than propagating. The element lookup and the assertion are now in separate blocks, so a genuine "no tours" condition reaches the caller as an `AssertionError`. Verified this doesn't change the passing-path test in `test_travel_gallery_smoke.py`, which already asserts on the returned count itself.

```
src/core/partners_base_api.py - _request_with_retry()
```

`ReadTimeout` appeared in two `except` tuples (the second unreachable) — removed. The `ConnectionError` / `ChunkedEncodingError` branch previously did not raise or return on the final attempt, silently yielding `None` (which `_request()` then turned into a generic, less informative `ValueError`, losing the original exception). It now re-raises the original exception once retries are exhausted, matching the `ReadTimeout` / `ConnectTimeout` branch's behavior.

FUNCTIONAL GAPS **Methods now honoring their parameters**

```
src/pages/ui/visa_enquiries_flow.py - fill_personal_details()
```

Accepted `first_name` / `last_name` / `email` / `phone` but always typed hardcoded values instead. Now fills the fields it's actually given. Confirmed existing callers in `test_visa_enquiries_smoke.py` already pass real (including intentionally empty, for partial-fill tests) values, so this fix makes those tests behave as their own comments already claimed.

```
src/pages/ui/visa_enquiries_flow.py - select_travel_date()
```

Computed `date_text` but never used it, always clicking the first enabled day past 10. Now tries to click the day matching `date_text` first, falling back to the old ">10" heuristic only if that exact day isn't available (e.g. disabled, or a different month is showing).

```
src/pages/ui/contact_flow.py - submit_form()
```

Accepted `max_retries` but implemented no retry loop. Now retries up to `max_retries` times before raising the last failure.

CODE QUALITY **Dead code & drift cleaned up**

```
src/pages/api/package_api.py - book_package()
```

Was defined twice with identical bodies; the duplicate (which was silently shadowing the first) has been removed.

```
src/pages/api/hotels_api.py
```

Removed a large commented-out legacy `search_hotels()` implementation, the verbose request/session monkey-patching debug logic in the live `search_hotels()`, and the unused `get_headers()` method (which built headers independently of `self.headers` and used a differently-cased key that no live request path read).

```
src/pages/ui/blogs_flow.py - navigate_to_blogs()
```

Removed an `except TimeoutError` clause that could never run, since the preceding `except Exception` always caught it first.

```
configs/environment.py - wait_for_api_environment() / override_token_extraction_config()
```

The former accepted `check_interval` but never used it (the wait was hardcoded); it now scales with the parameter. The latter silently dropped config updates for an unrecognized environment name; it now raises `ValueError` so a typo doesn't fail silently. No call sites pass an unrecognized environment today, so this is a safety net for future callers rather than a behavior change for existing ones.

```
src/pages/api/flight_api.py - get_search_results(), get_booked_flights(), get_quote()
```

Built query strings via raw f-string interpolation instead of a `params=` dict, so a value with spaces or special characters could produce a malformed request. All three now pass `params=`, matching the pattern used elsewhere in the API-client layer.

Stale header comments

`transactions_api.py`, `users_api.py`, and the three `partners_api/` files with a stray capital "P" in their leading path comment now match their actual filenames.

Every fix above was verified with `python -m py_compile` across all touched files, a full `pytest --collect-only` (158 tests collect cleanly, no import errors), and — for the `TestBase` reporting fix specifically — a throwaway passing/failing test pair run end-to-end to confirm real PASS/FAIL now surfaces correctly.

Remaining Recommendations for the Incoming Maintainer

A small number of items found during this pass were deliberately *not* changed, because fixing them means making a product/contract decision rather than correcting an unambiguous bug. They're listed here so they don't get lost.

`src/pages/ui/payment_flow.py` — `complete_payment_flow()`

OPEN Confirms the Flutterwave iframe loads but never fills in card details or submits a payment, despite unused `CARD_NUMBER_INPUT` / `EXPIRY_DATE_INPUT` / `CVV_INPUT` / `PAY_BUTTON` / `TEST_CARD_DETAILS` locators/data suggesting it was meant to. Left as-is because completing this would drive an actual (sandbox) payment gateway interaction — worth a deliberate decision on whether that's desired in CI, rather than a silent behavior change.

`src/pages/ui/flight_booking_flow.py`

OPEN Doesn't reuse `PaymentPage` or verify the Flutterwave checkout URL the way `package_booking_flow.py` does, and several passenger-form locators (passport, date of birth, country of origin) are defined but never filled. Left open for the same reason as above, plus the fact that the flight route is currently hardcoded (Heathrow→Schiphol) — extending this flow is a small feature-completion task, not a one-line fix.

`src/pages/api/visa_enquiries_api.py` — `get_visa_application_by_id()`

OPEN Passes the ID as a query parameter rather than a path segment, inconsistent with the `/_{id}` pattern used elsewhere. Left unchanged since altering the request shape could break against a backend that only accepts the current form — confirm with backend before changing.

`src/pages/api/partners_api/organization_api.py` — `reset_api_keys()`

OPEN Uses GET for what sounds like a mutating action (rotating/invalidating API keys). Left unchanged for the same reason — confirm the actual backend contract before switching to POST.

`src/core/base_api.py` / `src/core/partners_base_api.py`

OPEN Still two independent base classes duplicating session/retry/header plumbing. Their retry semantics are now aligned (Section 12), but consolidating them behind one shared ancestor remains a worthwhile — if larger — refactor.

Documentation coverage

OPEN `src/tests/` (~19 files) and the finer-grained internals of `src/utils/` (15 files) are referenced here but not documented method-by-method. Extending inline documentation to those areas remains a good next pass.

Suggested order of work: confirm the two API-contract questions (visa lookup, API-key reset) with backend first — they're quick to resolve either way. Then decide, as a team, whether the payment/flight-booking flows should be completed to actually exercise checkout end-to-end, or whether their current "verify the page loads, don't submit" scope is intentional and should just be documented as such in the test plan.